Deep Learning for Space, Time, Graph

Programming Assignment 2

Submitted to: Dr. Momin Uppal
Submitted by: Khawar Ali (25280094)

GITHUB: **khwrali011/DL4STG_PA2**

Question 1

For the committee. Every ten minutes the proposed system, **RiverST**, publishes for each of S1–S4 the expected water level one hour ahead and a nominal 80 % range (10th–90th percentile). It uses only what has been received by then: recent levels, rain at R1/R2, the reservoir's actual releases and the release plan already announced. Rain that has not yet fallen, inflow from the unmonitored tributary and unannounced operator actions cannot be known, so the range is part of the product; it widens when data are missing, and every forecast carries a data-quality flag. This is a design proposal: nothing here has been trained, and the range must be checked on held-out floods before use.

1. Architecture diagram

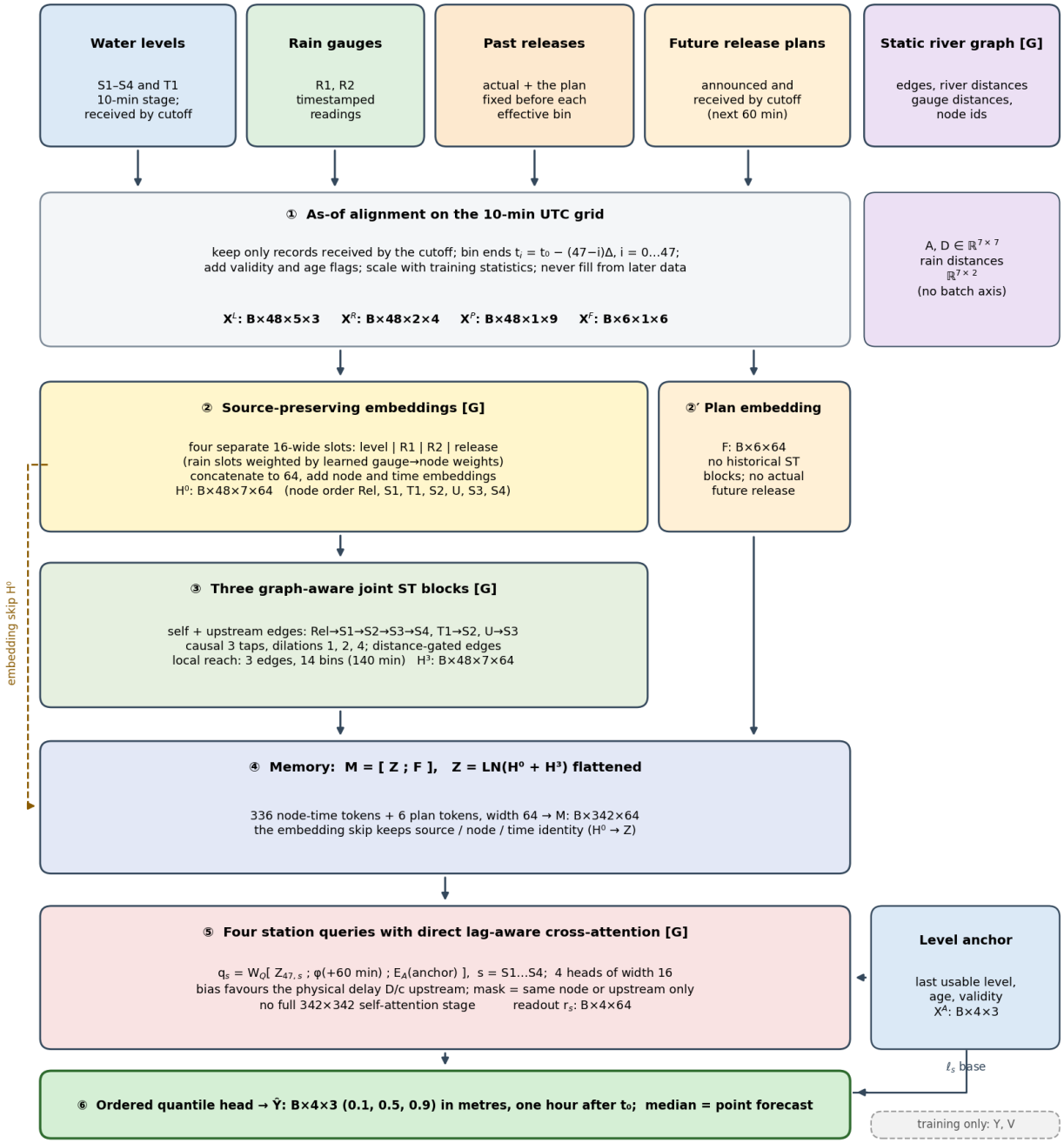


Figure Q1. *RiverST. B = batch; 48 = history bins (8 h); 7 = nodes [Rel, S1, T1, S2, U, S3, S4], with U a latent node for the unmonitored tributary; 64 = width. [G] marks blocks that use the static river graph. Alignment, projection and reshaping: the five gauged level series fill five of seven node slots; rain is projected from two gauges to seven nodes by learned weights; releases enter the Rel slot; (48, 7) is flattened to 336 tokens and the six future plan tokens are appended only after the historical blocks.*

2. Assumptions

Given: ten-minute levels at S1–S4 and T1; rain readings at R1, R2 with timestamps and locations; actual releases and announced schedules with announcement and effective times; river connectivity, locations and distances; a monitored tributary joins between S1 and S2 and an unmonitored one between S2 and S3; stations are unevenly spaced; exact travel times and the unmonitored inflow are not supplied; future rain and future levels are unavailable at issuance. **Everything below is a proposed working assumption** and each numerical value is to be checked on the authority's data.

ID	Assumption	Why reasonable	Effect on the design
A1	About 3 years of 10-min history (≈ 157 800 bins) containing several high-water events; split chronologically 70/15/15.	Typical gauge archives; chronological splits avoid leakage between overlapping windows.	Small model (order 105 parameters). If the record is shorter, cut width/blocks and state the weaker extreme-event evidence.
A2	About 98 % of level readings are present; gaps are mostly 1–3 bins, with rarer outages of hours. Most records arrive within 2 min of their timestamp, about 5 % up to 60 min late. Arrival times are logged.	Usual telemetry behaviour; logging is cheap.	Validity and age channels; 2-min grace period; as-of snapshots (Section 5). If logs are absent, simulate delays.
A3	Outages are not random: sensors fail more often during storms and floods.	Power, debris and communication faults cluster in bad weather.	Outage training includes high-water windows; missing-data results are reported separately for high stage.
A4	Clocks agree to about 1 min (UTC); units are consistent; gauge datums are fixed and documented.	Standard telemetry practice; needed to combine feeds at all.	Standardise per station on training data; flag unexplained level shifts.
A5	Rain readings can be turned into 10-min amounts; unknown rain is not zero rain. R1 represents the catchment nearer the reservoir and S1–S2, R2 the side nearer S2–S4 (from the project figure); rain is spatially uneven.	Matches gauge positions; two gauges cannot cover a mountain catchment.	Separate rain slots with validity; learned gauge-to-node weights started from distance; storms missing both gauges stay a limitation.
A6	Water moves downstream with a delay D/c that grows shorter at higher stage (celerity up to ≈ 2×faster). Working total delay from reservoir to S4 ≈2–6 h; single reaches ≈ 0.3–2 h.	Project statement says travel times vary with conditions; ranges are placeholders to be checked.	8 h history; stage-dependent delay prior; the longest delay is not a guaranteed bound, so longer windows are tested.
A7	Downstream influence dominates; backwater (e.g. at the town bridge near S4) and unrecorded withdrawals are weak; inflows add at junctions.	Matches the described system.	Edges point downstream only; reverse effects are a stated limitation.
A8	U is a latent confluence node: rain-driven plus slow base flow, located mid-way between S2 and S3 unless told otherwise.	It is an ordinary tributary; its past effect is visible in S3.	Node U gets a learned embedding and rain features; its state is not an observed flow.
A9	Release plans are normally announced at least one hour ahead and may be revised, partial or missing; actual releases usually stay within ≈ 10 % of plan, with rare overrides.	Planned operations; both plan and actual are recorded, so deviation can be learned.	Six future plan tokens with availability, revision and fallback flags.
A10	Relations are stable enough to learn from the training period but drift slowly (season, channel change).	Typical for a few years of data.	Chronological validation, drift monitoring, retraining after sustained degradation.
A11	One CPU service runs the 10-min cycle; feeds are ready by t0 + 2 min.	Modest requirement.	Model kept small; latency to be measured (Section 5).
A12	The authority supplies warning stages later; the model forecasts level in metres, not discharge.	Rating curves are not provided.	No discharge conversion; thresholds only used to report results.

River dynamics assumed. One-way downstream routing with a stage-dependent delay, additive inflow at two junctions, catchment rainfall raising levels after a delay, and release changes reaching S1 after a short delay. These are encoded as graph structure and as the initial value of a delay prior; their magnitudes are learned and nothing enforces mass conservation.

3. Forecasting task and data specification

Origin and cutoff. The forecast origin t_0 lies on the 10-min UTC grid; Δ = 10 min. The forecast is issued after a 2-min grace period, using records received by the cutoff t_0 + 2 min (A2, A11); training reproduces exactly this cutoff. **History.** H = 48 bins with endpoints $t_i = t_0 - (47 - i)\Delta$, i = 0…47; the first and last endpoints are 470 min apart. Five extra bins before the window supply 60-min rain totals. **Target.** The level at S1–S4 at t_0 + 60 min. **Outputs.** 10 %, 50 % and 90 % levels for each station; the median is the point forecast.

Item	Tensor	Axes (in order) and channels	Time coverage / availability
Levels (input)	X_L: B×48×5×3	batch; bin i; (S1,S2,S3,S4,T1); standardised level, validity, age	Latest usable reading in $(t_i - \Delta, t_i]$; never filled from later data.
Rain (input)	X_R: B×48×2×4	batch; bin; (R1,R2); 10-min amount, 60-min amount, validity of each	Completed intervals ending at t_i; the 60-min total needs six valid bins.
Past releases (input)	X_P: B×48×1×9	batch; bin; Rel; actual, planned, deviation e, 3-bin mean $\bar{e}$, valid(actual), valid(plan), valid($\bar{e}$), actual-age, announcement lead	Actual averaged over the bin; the plan is the version received before the bin began.
Future plans (input)	X_F: B×6×1×6	batch; future bin k = 1…6; Rel; planned release, valid, lead, revision, valid(revision), fallback-age	Bins ending t_0+10…t_0+60 min; latest version received by the cutoff.
Graph (input)	A, D: 7×7; ids: 7; gauge distances: 7×2	node × node (connectivity, river km); node; node ×rain gauge	Static; no batch or time axis.
Level anchor (input)	X_A: B×4×3	batch; (S1…S4); last usable level, age, validity	Last usable level within the history; training mean if none.
Forecast (output)	$\hat{Y}$: B×4×3	batch; station; quantile (0.1, 0.5, 0.9), metres	All four stations at t_0 + 60 min; point forecast $\hat{Y}[:, :, 1]$.
Targets (training only)	Y, V: B×4 each	batch; station; true level and usable-target flag	At t_0 + 60 min; never an input.

Missing values. Level checks (range, jump, flat-line) mark faulty readings missing. A missing level is carried forward only as a flagged imputation (validity 0, age recorded); with no past value it is the training mean with an age beyond the window. Missing rain is stored as zero after scaling with validity 0; genuine zero rain has validity 1. Where a plan is absent or does not cover a whole bin, the fallback is the last usable actual release (validity 0, fallback-age recorded), else the training mean with a severe-degradation flag; these are estimates, not observations.

Alignment. *Across locations:* equal bin endpoints mean equal clock time, and measured series are not shifted to remove delays, so a rise at S1 and its later arrival at S3 stay at different times and the model must connect them. *Across sampling times:* levels use the latest reading in the bin, rain uses completed amounts, releases use the bin average, and plans are indexed by *effective* time. **Available at issuance:** records received by the cutoff, plan versions received by the cutoff, static geometry. **Not available:** later arrivals, any future level, rain or actual release, later plan revisions, labels. Scalers and imputation values use the training split only; each window, its warm-up, anchor and target stay inside one split.

4. Model computations and design justification

Notation. d = 64; N = 7 nodes in the order [Rel, S1, T1, S2, U, S3, S4]; B is batch size; i indexes bins and increases toward t_0 (i = 47 is the latest bin); Pa(n) are the immediate upstream neighbours of node n (edges Rel→S1, S1→S2, T1→S2, S2→S3, U→S3, S3→S4); G is the set of five gauged level nodes; LN is per-token layer normalisation; ε is a small positive constant. Every layer has its own learned parameters.

(a) Source-preserving embedding. Each source gets its own 16-wide slot; slots are concatenated, never averaged, so a later block can still tell level from rain from release (notebook, Think about 2).

$$H^0_{i,n} = \left[L_{i,n};\, \omega_{n1} E_{R1}\left(x^R_{i,1}\right);\, \omega_{n2} E_{R2}\left(x^R_{i,2}\right);\, P_{i,n}\right] + e_n + \phi(t_i - t_0)$$

$$L_{i,n} = \mathbf{1}[n \in G] E_L\left(x^L_{i,g(n)}\right)$$

$$P_{i,n} = \mathbf{1}[n = \text{Rel}] E_P\left(x^P_i\right)$$

$$\omega_{nk} = \text{softmax}_k\left(\theta_{nk} - \gamma d^{\text{rain}}_{nk}\right), \qquad \gamma \geq 0$$

$$F_k = E_F\left(x^F_k\right) + e_{\text{Rel}} + \phi(k\Delta), \qquad k = 1, \ldots, 6$$

E_L maps 3→16, E_{R1}, E_{R2} map 4→16, E_P maps 9→16, E_F maps 6→64; e_n (node) and φ (time offset) have width 64. Absent sources (no gauge at Rel or U) give a zero slot; U receives only rain and its node embedding. H^0 is B×48×7×64 and F is B×6×64.

(b) Graph-aware joint space–time blocks (×3, dilations δt = 1, 2, 4). Each block updates a token from itself and its upstream neighbours over three causal taps, with a separate weight matrix for every tap (as in the notebook's ST block) and neighbours defined by river edges, not by adjacent rows:

$$J_{i,n}^{\ell} = \sum_{r=0}^{2} W_{0r}^{\ell}\, H_{i-r\delta_\ell,n}^{\ell} + \sum_{m\in \mathrm{Pa}(n)} \sum_{r=0}^{2} g_{i,nm,r}^{\ell}\, W_{1r}^{\ell} H_{i-r\delta_\ell,m}^{\ell}$$

$$g_{i,nm,r}^{\ell} = \mathrm{sigmoid}\left(u_r^{\ell T}\left[H_{i-r\delta_\ell,n}^{\ell}; H_{i-r\delta_\ell,m}^{\ell}; \psi(D_{mn})\right]\right)$$

$$H_{i,n}^{\ell+1} = \mathrm{LN}\left(H_{i,n}^{\ell} + \mathrm{GELU}\left(J_{i,n}^{\ell}\right)\right)$$

$$Z = \mathrm{LN}(H^0 + H^3), \qquad M = [Z; F]$$

Taps before bin 0 are zero padding, not measurements. ψ embeds river distance D_{mn} (km), so reaches of different length get different gates. Z is flattened to 336 tokens and M is B×342×64. **Local reach:** 3 edges and 1 + 2(1 + 2 + 4) = 15 bins (maximum lag 14 bins = 140 min). This cannot cover the four-edge Rel→S4 path or a six-hour delay, which is why the forecast does not read only from local states.

(c) Direct, lag-aware forecast attention. One query per station for the single required horizon, built from that station's latest token, a fixed +60 min embedding and its level anchor, attends over *all* permitted memory tokens:

$$q_s = W_Q\left[Z_{47,s}; \phi(60\ \mathrm{min}); E_A(x_s^A)\right]$$

$$\alpha_{s,j}^{a} = \mathrm{softmax}_j\left(\frac{\left(W_q^a q_s\right)^T \left(W_k^a M_j\right)}{\sqrt{16}} + b_{s,j}^{a} + \Lambda_{s,j}\right)$$

$$b_{s,j}^{a} = -\mathbf{1}\left[n_j \neq s\right]\frac{\left(\Delta t_{s,j} - D_{s,j}/c_s^a\right)^2}{2\tau_a^2} + \beta_a\left(\mathrm{bucket}\left(\Delta t_{s,j}\right), \mathrm{rel}(s,j)\right)$$

$$c_s^a = \varepsilon + \mathrm{softplus}(\theta_a + \kappa_a z_s)$$

$$\tau_a = \varepsilon + \mathrm{softplus}(u_a)$$

Four heads of width 16. $\Delta t_{s,j}$ is the target time minus the token time (hours); $D_{s,j}$ is the river distance (km) from the token's node to station s (0 for the same node); z_s is the station's latest admissible standardised level (0 with validity 0 if none), never the unknown target. The delay prior applies only to upstream tokens; same-node tokens use the learned recency offset β_a, so a station is not pushed toward lag 0. The mask $\Lambda_{s,j}$ is 0 for tokens at the station itself or upstream of it and $-\infty$ otherwise (S1: Rel; S2: Rel, S1, T1; S3: also S2, U; S4: also S3). Future plan tokens belong to Rel and are always allowed. θ_a is initialised from D(Rel, S4)/6 h; all parameters are learned, and the bias is a soft routing preference, not a measured speed. A missing source does not mask a fused token: its validity flags are inside the token.

$$r_s = \mathrm{LN}\left(q_s + W_O \mathrm{Concat}_a\left(\sum_j \alpha_{s,j}^{a}\, W_V^a M_j\right)\right)$$

$$\tilde{r}_s = \mathrm{LN}\left(r_s + \mathrm{FFN}(r_s)\right)$$

(d) Forecast mapping with ordered quantiles. A station-specific 64→32→3 MLP gives (m_s, l_s, u_s); ℓ_s is the anchor level (training mean if none) and σ_s the training level scale. Zero median change is persistence:

$$(m_s, l_s, u_s) = \mathrm{MLP}_s(\tilde{r}_s)$$

$$\hat{y}_s^{0.5} = \ell_s + \sigma_s m_s$$

$$\hat{y}_s^{0.1} = \hat{y}_s^{0.5} - \sigma_s \mathrm{softplus}(l_s)$$

$$\hat{y}_s^{0.9} = \hat{y}_s^{0.5} + \sigma_s \mathrm{softplus}(u_s)$$

(e) Training objective. Pinball loss at q ∈ {0.1, 0.5, 0.9} on scaled error, masked by the usable-target flag V_{bs} (batches with no valid target are skipped):

$$\rho_q(v) = \max(qv, (q-1)v)$$

$$\mathcal{L} = \frac{\sum_{b,s} V_{bs} \sum_{q\in\{0.1,0.5,0.9\}} \rho_q\left(\frac{y_{bs} - \hat{y}_{bs}^{q}}{\sigma_s}\right)}{3\sum_{b,s} V_{bs}}$$

The loss is *not* weighted by the unknown future level: selecting or weighting by the outcome would change the distribution being fitted and bias the quantiles. Emphasis on high water instead comes from oversampling windows whose *input* stage is high (a selection on inputs leaves the conditional quantiles unchanged), and from reporting high-water results separately. AdamW, learning rate 10^{-3}, batch 64, validation-based early stopping (provisional); one-third of the validation segment is reserved for interval calibration (Section 6).

Design justification

Notebook finding (saved results)	Decision in RiverST
T-only blocks cannot see a neighbour: on the moving field its test MAE is ≈ 0.502, the same as predicting zero (0.502).	A forecast must receive the upstream sources that cause it; deeper local temporal stacks are not the remedy. Edges Rel→S1…S3→S4, T1→S2, U→S3 are explicit.
With identical local histories and targets −0.594 and 0.900, MAE is minimal (0.747) for every p in [−0.594, 0.900] and MSE is minimal only at the mean 0.153 (0.558), a value that matches neither target.	Missing information means a single number can look right and be wrong. Output median plus 10/90 % quantiles with pinball loss and report coverage; do not train on squared error.
S→T and T→S depend on what the first block preserves (who saw the signal first, and when).	Separate source slots, node and time embeddings, and an embedding skip ($Z = \mathrm{LN}(H_0 + H_3)$); no early averaging over sources or time.
ST and an alternating pair have the same reach but learn different functions; neighbours on a line are not river neighbours.	Graph-aware joint ST blocks on river edges, per-tap weights, distance-gated. Planned ablation: graph S→T, T→S and joint ST with equal inputs, width and readout.
Depth: an observation 4 sensors away and 6 steps old needs max(4,⌈6/2⌉) = 4 alternating pairs or 4 ST blocks.	Here Rel→S4 is 4 edges and a 6 h delay is 36 bins, i.e. ≥ 18 ordinary 3-tap blocks in time. Local blocks stay shallow (3, dilated); long paths come from the readout.
Relay world, 3 ST blocks: near MAE 0.0069, far 0.5004 (zero-forecast baseline 0.498); 3 attention blocks: 0.0011 near and far. Every head reads only its own latest state.	Local readout is the bottleneck. Station queries cross-attend to all permitted tokens. The relay is an instantaneous broadcast, not a river law, so only the access lesson is borrowed.
Attention costs $C_A = 2L^2d$ versus $C_{ST} = 9Ld^2$ ($L = 128$, $d = 32$: 1.05 M vs 1.18 M per ST block; 15 ST blocks to reach every distance: 17.7 M). One head per run takes the source; which one varies.	Four queries instead of full self-attention: $C = 2QLd$ = 175 104 versus $2L^2d$ = 14 971 392 at $L = 342$, $d = 64$. Attention maps alone are not trusted: perturbation tests and ablations are planned (Section 6).

Why each forecast can use the right information. A station's 60-min forecast needs upstream conditions at (target time −delay). If the delay is at least 60 min, those conditions are already observed or in transit; if it is shorter, they have not happened yet, and only the release plan (and not future rain) anticipates them. That is why the plan tokens exist and why S1 depends so strongly on them (A6, A9).

Station	Main drivers of the one-hour forecast	Where RiverST gets them
S1	Reservoir release (actual and planned), local runoff from R1, its own level.	Rel history and future plan tokens through the Rel→S1 edge and the delay prior; R1 through learned gauge weights.
S2	S1 routed with delay, T1 routed with delay, local rain, own level.	S1 and T1 tokens at lag ≈ D/c; both upstream branches stay separate; R1/R2 slots.
S3	S2 routed, the unmonitored tributary U (rain-driven, partly inferred from S3's own history), local rain.	S2 tokens; node U with rain features; the gap between S3 history and routed S2 is learned through attention.
S4	S3 routed (short delay) and, at high stage, earlier S2, S1 and release conditions.	Direct paths over the whole four-edge route and the full 8 h window; stage-dependent speed c.

Physical versus learned.*Imposed:* topology and flow direction, information cutoff, additive-inflow structure, a positive delay scale initialised from distance, ordered quantiles. *Learned:* feature transformations, gauge-to-node rain weights, edge gates, how delay depends on stage, release reliability, the level response and the interval width. No water-balance equation, rating curve or observed tributary discharge is imposed, and the optional separate simulator is not used.

5. Operation and exceptional conditions

(i) Missing observations

Sensor-specific range, jump and flat-line checks reject faulty values; the reason is logged. The token keeps its slot with validity 0, a flagged carried-forward value and its age, and other valid sources at that node are kept. Training simulates random gaps, bursts of 1–6 h and whole-station outages, including during high water (A3); masks, ages and deviation features are recomputed after each simulated gap. A station forecast is marked degraded when its anchor is older than one hour (threshold to be validated). If no usable dynamic input remains, the output is a flagged persistence or training-prior fallback and its interval is not called calibrated.

(ii) Delayed observations

Each forecast uses a frozen as-of snapshot: a reading measured earlier but received after the cutoff is unavailable to that forecast. It is inserted at its original measurement time in the next cycle if still inside the window; issued forecasts are never revised. Input versions and both timestamps are logged. Training reconstructs the same snapshots from arrival logs, or from delays simulated with the A2 distribution. Late labels may be used for later training or scoring, never as earlier inputs.

(iii) Announced future releases

For each of the six future bins the latest plan version received by the cutoff is selected and aligned by effective time; its revision is the difference from the previous version for that bin. These six tokens are kept out of the historical blocks and enter only the memory. Actual future releases are

never used, in training or operation. Missing or partial plans use the flagged persistence fallback of Section 3, and such cases are included in training (including by randomly removing plans) so the fallback is calibrated. Plans are known inputs, not guarantees of action.

(iv) Scheduled versus actual releases

Past tokens carry the actual rate, the plan in force, their deviation e_i = actual − plan and its three-bin mean $\bar{e}_i = (e_i + e_{i-1} + e_{i-2})/3$ (valid only when all three are). The readout combines these with the future plan, so persistent deviations shift the effective release and widen the interval; nothing uses an actual release after the cutoff. An operator alert is raised when three consecutive valid bins exceed $\max(\delta_{abs}, 0.15\,|\text{plan}|)$, with δ_{abs} set from meter resolution or authority tolerance (15 % is an initial choice). Unannounced emergency actions remain unpredictable from the supplied inputs.

Feasibility every ten minutes

The service keeps rolling buffers, adds new records, validates them, runs one inference, and publishes four medians with intervals, origin and target times and quality flags; retraining and calibration are offline, and a flagged fallback is served if inference fails. With $L = 48 \times 7 + 6 = 342$, $Q = 4$, $d = 64$, the readout mixing count is $C_{readout} = 2QLd = 175\,104$, against $2L^2d = 14\,971\,392$ for full self-attention over the same memory. The local blocks need about 3 blocks × 3 taps × 48 bins × (7 self + 6 edge terms) × $64^2 \approx 23.0$ million multiply-accumulates. These are partial analytical counts (they leave out projections, normalisation, the feed-forward layer and data handling), not latency measurements; they indicate a modest compute load against a 600 s cycle, and the full implementation must be timed before deployment.

6. Uncertainty, limitations, and additional information

The intervals describe uncertainty conditional on the available inputs. They do not make unseen storms, unknown inflows or unannounced operator actions observable.

Limitation	Forecast impact	Response / what remains unresolved
Unmonitored tributary U	S3 and S4 can rise without a matching S2 rise.	Latent node U with rain features; S3 and S4 assessed separately; wider spread in rain. **Unresolved:** U's flow is not uniquely identified and its future inflow is unknown.
Changing, longer travel times	Peaks arrive early or late; relevant history may fall outside 8 h.	Stage-dependent delay, learned lag offsets, longer-window tests. **Unresolved:** floodplain storage can break the initial delay assumption.
Future rain unknown; two gauges	A new storm, or one missing both gauges, produces an unanticipated rise.	Observed rain only; wet-period coverage checked. **Unresolved** without rain forecasts or radar.
Plan deviations and overrides	Unexpected release changes hit S1 and downstream.	Deviation and revision features, alerts, outage-style training. **Unresolved:** truly unannounced emergencies.
Faults, delays, datum shifts	Stale or biased inputs shift forecasts.	QC, validity and age flags, as-of evaluation, maintenance records. **Unresolved:** slow undetected bias.
Rare floods, drift	Average accuracy may not carry to extremes or changed conditions.	High-stage and event results reported separately; drift monitoring. Good average error is not evidence of flood safety.
Backwater, withdrawals	Upstream-only dependence may be wrong.	Request records; revise graph with evidence. **Unresolved:** strong reverse effects are outside the model.
Calibration; unused paths	Intervals mis-sized; a connected source may be ignored.	Coverage and width by station and condition; perturbation tests and ablations below.

Request to the authority	How it helps	Usable without it?
Rain forecasts, radar, or more gauges	Shows approaching rain and spatial variation missed by R1/R2.	Yes: observed rain only; future and local storms unresolved.
Tributary gauge, confluence position, catchment area and land cover	Constrains the missing tributary and its rain response.	Yes (A8): U stays latent, its location uncertain.
Observed wave travel times, channel geometry, hydraulic studies	Better delay prior and storage/backwater information.	Yes (A6–A7): learned delays stay approximate.
Reservoir level, gate state, operating and override rules	Explains departures from announced releases.	Yes: recent deviations help; unseen overrides remain.
Arrival logs, sensor QC and maintenance logs, clock, unit and datum metadata	Realistic as-of training; fault detection.	Logs optional (simulate delays); reliable clock, unit and datum alignment is essential.
Rating curves and official station warning levels	Relate level to discharge; judge decision-relevant floods.	Yes: forecast level directly; a training percentile is only a diagnostic.
Soil moisture, snow and temperature, withdrawals, downstream controls	Explain omitted runoff, storage and reverse influences.	Yes, within the assumptions.

Checks before and after training

Reach before training. Audit tensor connections and masks so that every permitted upstream token has a path to each station query (the notebook's reach map, applied to the river graph).

Use after training. Hold the receiver history fixed, perturb one upstream measurement or planned release over a plausible range, recompute dependent features, and check the lag at which S2–S4 respond. A constant output is evidence to investigate; a response alone does not prove a correct physical effect. Inspect all heads and runs, since the head that takes the source varies between runs.

Ablations. Local stack with local readout versus the full readout; remove one upstream source; remove the delay prior; compare history lengths; compare graph S→T, T→S and joint ST. Keep evaluation cases fixed.

Baselines and metrics. Persistence and a station-only temporal model; MAE and RMSE of the median, pinball loss, interval coverage and width per station, with high-water, missing-data and wet-period subsets, on chronological periods or whole events.

Interval calibration. If needed, fit a station-specific widening on the reserved calibration segment without moving the median, and call the result a calibrated interval, not exact 10th/90th quantiles. Dependence in time and few extreme events prevent an unconditional coverage guarantee; recheck after drift.

Question 2

• All 10,000 Moving MNIST videos were loaded with split=None. Dataset indices 0–7,999 were used for training, 8,000–8,999 for validation, and 9,000–9,999 for testing. Pixel values were converted to float32 and divided by 255.

• Each input contains five frames. Every model predicts five future frames using the same nine-step autoregressive rollout. True future frames are never fed back; gradients flow through predicted feedback during training.

• A–D were each trained for 30 epochs with seeds 0 and 1, Adam, learning rate 0.001, batch size 32 and no learning-rate schedule. Final-epoch weights were retained. One epoch samples one window from each of the 8,000 training videos.

• Two Tesla T4 GPUs were used, with one seed per GPU. The architecture of D was fixed before test evaluation. No test-set model selection was used. All performance results use the 1,000 test videos at start frame t = 1; all examples and histograms use seed 0.

At speed k, inputs are frames t, t+k, …, t+4k. Available targets are t+5k, …, t+9k, limited to frame 20. A/B train with k=1 and t=1,…,11. C/D sample k uniformly from {1,2,3}, then t uniformly from {1,…,14}, {1,…,8}, or {1,2}, respectively. Unavailable targets are excluded from loss and evaluation.

The training loss first averages squared error over pixels and available horizons within each video, then averages over videos. Therefore a two-target video and a five-target video receive equal weight. $E_{(5)}$ averages MSE over five horizons; $E_{(2)}$ averages the first two. Cross-speed comparisons use $E_{(2)}$.

Table 1. Transformation and temporal-window checks

The checks use torch.manual_seed(0) followed by a random tensor of shape (20,1,64,64), and a zero marker video with p[7,0,10,20]=1. Rows (a)–(c) report maximum absolute difference. Window checks use a video filled with its one-based frame numbers.

Check	Expected	Measured
(a) M(M(x)) vs x	0	0.0
(b) $R^4(x)$ vs x	0	0.0
(c) MRM(x) vs $R^{-1}(x)$	0	0.0
(d) Marker after M	(7, 10, 43)	(7, 10, 43)
(e) Marker after R	(7, 43, 10)	(7, 43, 10)
(f) 1× window	([1, 2, 3, 4, 5], [6, 7, 8, 9, 10])	([1, 2, 3, 4, 5], [6, 7, 8, 9, 10])
(g) 2× window	([1, 3, 5, 7, 9], [11, 13, 15, 17, 19])	([1, 3, 5, 7, 9], [11, 13, 15, 17, 19])
(h) 3× window	([1, 4, 7, 10, 13], [16, 19])	([1, 4, 7, 10, 13], [16, 19])

The marker uses zero-based indices; frame selections in (f)–(h) use the handout's one-based frame numbering.

1. Training-data motion statistics

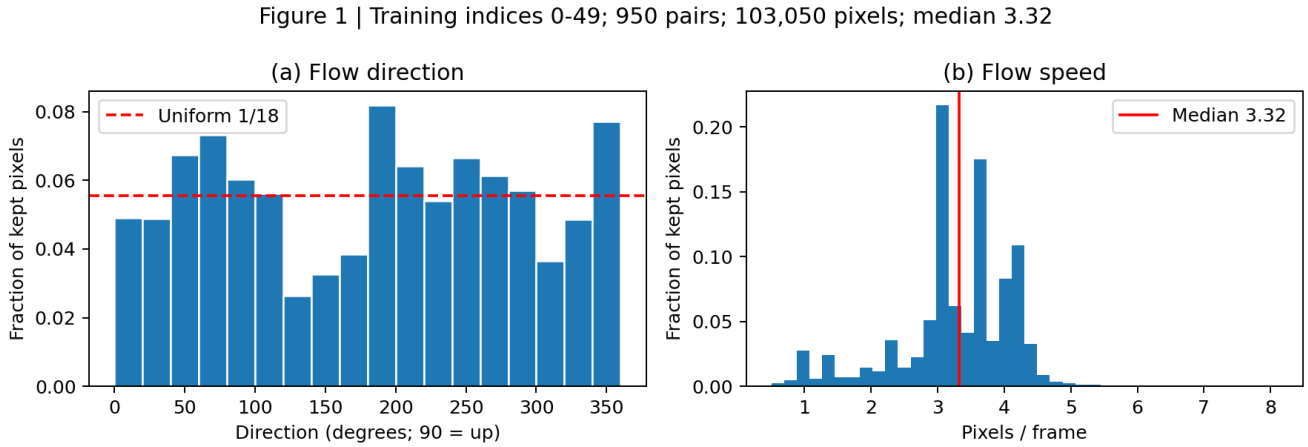


Figure 1. Optical flow from training videos 0–49: 950 consecutive-frame pairs and 103,050 retained pixels. Median estimated speed: 3.32 pixels/frame. Direction bins are 20° wide; the dashed line is 1/18. Each histogram reports fractions of retained pixels.

Flow was estimated on uint8 frames with Farneback parameters (0.5, 3, 15, 3, 5, 1.2, 0). Pixels were retained only when intensity exceeded 51 in both frames and flow magnitude exceeded 0.5. Direction was atan2(−dy, dx), mapped to [0°,360°), so 90° points upward.

Q2.1

Largely yes. All 18 direction bins are populated in Figure 1(a): the fractions run from 0.0263 to 0.0820 against the uniform value 1/18 = 0.0556, so no direction is missing from training. Mirroring sends a direction θ to 180° − θ, which reflects the histogram about the vertical axis; a 90° anticlockwise rotation sends θ to θ + 90°, which shifts it by 4.5 bins (not a whole number of 20° bins, so the exact rotated histogram would need re-binning). If every training video were mirrored or rotated, the histogram would be reflected or shifted, and it would look the same only if the original were symmetric. It is not exactly symmetric, but every direction a transformed video can show is already common in training, so the mirror and rotation tests mainly present unfamiliar digit appearance, not unfamiliar motion directions. The unevenness is about what 50 videos can show: they contain only about 100 digit trajectories, roughly 5–6 per bin, and Poisson fluctuations of ±2.4 trajectories (±43%) would by themselves spread the bin fractions from about 0.03 to 0.08, which is what Figure 1(a) shows. This is a back-of-the-envelope check, not a significance test.

From Figure 1(b), a typical speed is the median, 3.32 pixels per frame; most pixels lie between about 3 and 4.4 px/frame, in clusters near 3.0, 3.6 and 4.2, and almost none exceed 5.4. A 2× clip therefore moves about 2 × 3.32 ≈ 6.6 px per sampled step (roughly 6–8.4 px for the main clusters) and a 3× clip about 3 × 3.32 ≈ 10 px (roughly 9–12.6 px), far above any 1× training step. These are scaled local-flow estimates, not measured net displacements: a wall bounce between sampled frames shortens the net displacement.

Overlap and the flow mask. Optical flow is unreliable where digits overlap, and requiring intensity above 51 in both frames favours pixels that stay inside a digit and underrepresents the largest displacements. This is not enough to change the answer. Overlaps affect only the frames in which the two digits cross, and estimation error smears directions rather than emptying a bin that holds about 2.6% of the 103,050 retained pixels (roughly 2,700), so the finding that all directions are present stands. What the bias does limit is precision: the exact bin fractions, the exact median, and any claim that the true direction distribution is uniform or exactly symmetric.

Q2.2

Each transformation, in turn. Mirror changes both the motion statistics and the appearance: it reflects every direction (θ → 180° − θ) and reverses the digits left to right. Rotation changes both as well: it turns every direction by 90° and lays the digits on their side, a larger change of appearance. Both keep speeds unchanged and the motion valid (constant velocity with elastic bounces in a square). Speed-up changes only the motion: appearance is untouched, but the step between sampled frames is k times larger and a bounce can fall between them.

Why this is a controlled experiment. Figure 1(a) shows that the directions produced by mirror and rotation are already in the training data, so those two tests mostly vary appearance while the rule that generates the motion is unchanged; speed-up varies the motion statistics (and the distance information must travel) while appearance stays the same. A failure under one test but not another therefore points to a specific cause. The separation is not perfect, because the training direction histogram is not exactly symmetric.

2. Baseline forecasting

Table 2. Normal-speed accuracy and parameter count

Metric	A	Blank	Copy-last
MSE_1	0.0169 (0.0173, 0.0166)	0.0426	0.0492
MSE_2	0.0205 (0.0208, 0.0202)	0.0426	0.0604
MSE_3	0.0228 (0.0231, 0.0224)	0.0426	0.0669
MSE_4	0.0245 (0.0248, 0.0242)	0.0425	0.0708
MSE_5	0.0261 (0.0264, 0.0258)	0.0425	0.0732
$E^{(5)}$, 1×	0.0222 (0.0225, 0.0218)	0.0426	0.0641
Parameters	480784	0	0

Learned-model entries: mean (seed 0, seed 1). Parameter counts are exact and identical across seeds. Blank and Copy-last are deterministic single values.

Q2.3

Pixel unshuffle folds each 4×4 patch into channels. One folded-grid step corresponds to 4 original pixels centre-to-centre. A 3×3 convolution reaches one neighbouring folded cell per axis. The longest path within one update is new input → layer 1 → layer 2 → output, or previous layer-1 hidden state → layer 1 →layer 2 → output. Both traverse two spatial convolutions.

Reach of A = 2 layers × 1 cell × 4 pixels = 8 pixels per axis. This is a Chebyshev-distance bound; the farthest diagonal corner is $8\sqrt{2} \approx 11.3$ pixels away in Euclidean distance. Previous layer-2 state traverses only one convolution. The four gates are parallel, not four serial spatial layers. Elementwise cell updates, the 1×1 head and fold/unfold operations add no reach under the stated convention. Earlier recurrent steps increase accumulated context but do not increase this one-update bound.

Q2.4

A's mean $E^{(5)}$ is 0.0222, compared with 0.0426 for Blank and 0.0641 for Copy-last. Relative to these references, A reduces error by 47.9% and 65.4%, respectively. It therefore predicts useful information beyond a static or empty forecast.

Error increases monotonically across horizons for both seeds. The mean rises from 0.0169 at horizon 1 to 0.0261 at horizon 5, an increase of about 54%. Figure 2 shows growing blur as the rollout continues. In video 9001, the true 8 and 2 overlap and then separate; A blends their appearances and does not recover two clear digit identities. The low MSE of Blank also shows why background-dominated pixel error must be interpreted alongside images.

Q2.5

The following is the pre-experiment prediction.

```I expect A to be imperfect under both mirror and rotation, with rotation usually harder because upright digit appearance is absent after rotation. The measured flow median is 3.3175 pixels/frame; 2x and 3x correspond to approximately 6.635 and 9.95251. A has a one-step reach of 8 pixels per axis (8 sqrt(2) along a diagonal); I expect worsening at higher speeds, especially when displacement components exceed 8 pixels, but this is a hypothesis, not a hard speed cutoff.```
```

Figure 2. Baseline prediction examples

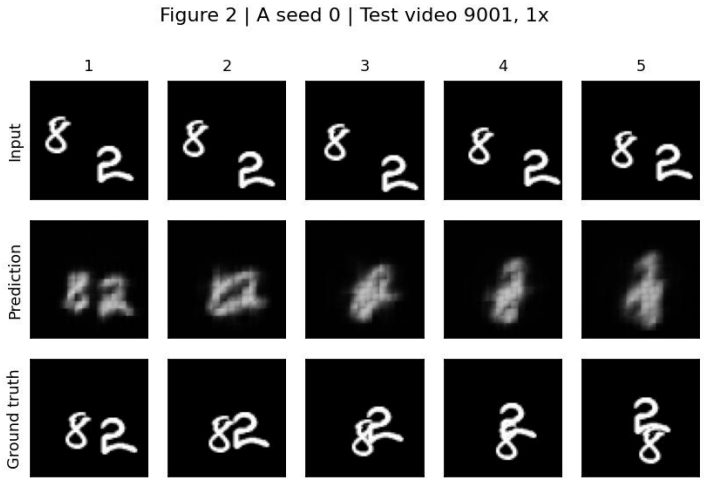


Figure 2(a). Test video 9001, model A, seed 0, 1×. Top: input frames 1–5. Middle: predictions for frames 6–10. Bottom: true frames 6–10. This example contains overlapping digits.

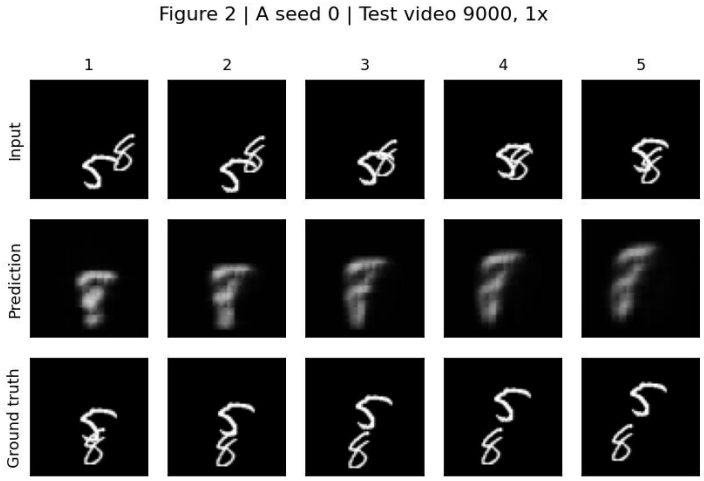


Figure 2(b). Test video 9000, model A, seed 0, 1×. Top: input frames 1–5. Middle: predictions for frames 6–10. Bottom: true frames 6–10.

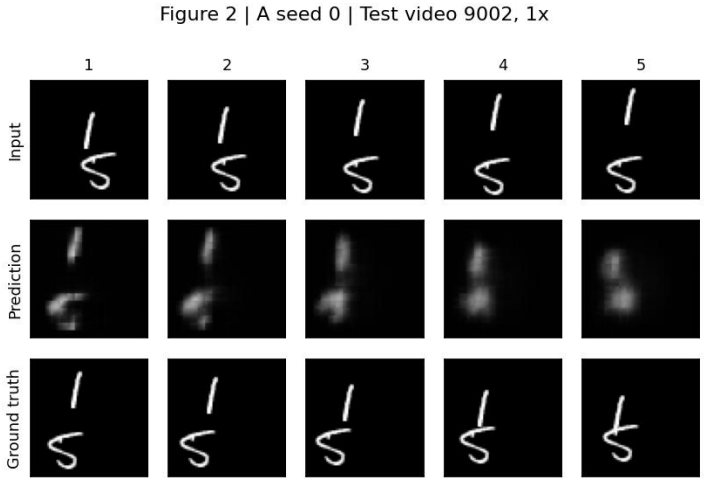


Figure 2(c). Test video 9002, model A, seed 0, 1×. Top: input frames 1–5. Middle: predictions for frames 6–10. Bottom: true frames 6–10.

3. Mirror and rotation stress tests

Table 3. Consistency together with transformed accuracy

Metric	A	Blank	Copy-last
$E^{(5)}$, 1×	0.0222 (0.0225, 0.0218)	0.0426	0.0641
C_m	0.00197 (0.00195, 0.00199)	0	0
E_m	0.0228 (0.0230, 0.0225)	0.0426	0.0641
C_r	0.00316 (0.00317, 0.00315)	0	0
E_r	0.0239 (0.0241, 0.0236)	0.0426	0.0641

Learned-model entries: mean (seed 0, seed 1). Blank and Copy-last are deterministic, so a single value is shown.

For T=M or R, C_T = MSE[f(TX), T f(X)] and E_T = MSE[f(TX), TY]. Each is averaged over pixels, five horizons and all test videos at 1×. A lower consistency gap alone does not establish correctness.

Figure 3 shows the two largest C_M cases (9613 and 9765) and the two largest C_R cases (9279 and 9884), selected from A seed 0. Columns 1–5 are inputs; columns 6–10 are future frames. The four rows are [X, f(X)], [TX, TY], [TX, f(TX)] and [TX, T f(X)]. Figure 4 follows the Figure 3 panels and shows where these four videos sit in the per-video distributions of CM and CR.

Q2.6

Blank and Copy-last have exactly zero mirror and rotation gaps. Zero images are unchanged by pixel permutations, and copying the last input commutes with applying the same permutation to each frame.

Nevertheless, both have substantially higher transformed errors than A. Consistency measures agreement between routes, not agreement with the true future. E_T must therefore accompany every C_T.

Figure 3. Largest mirror inconsistencies

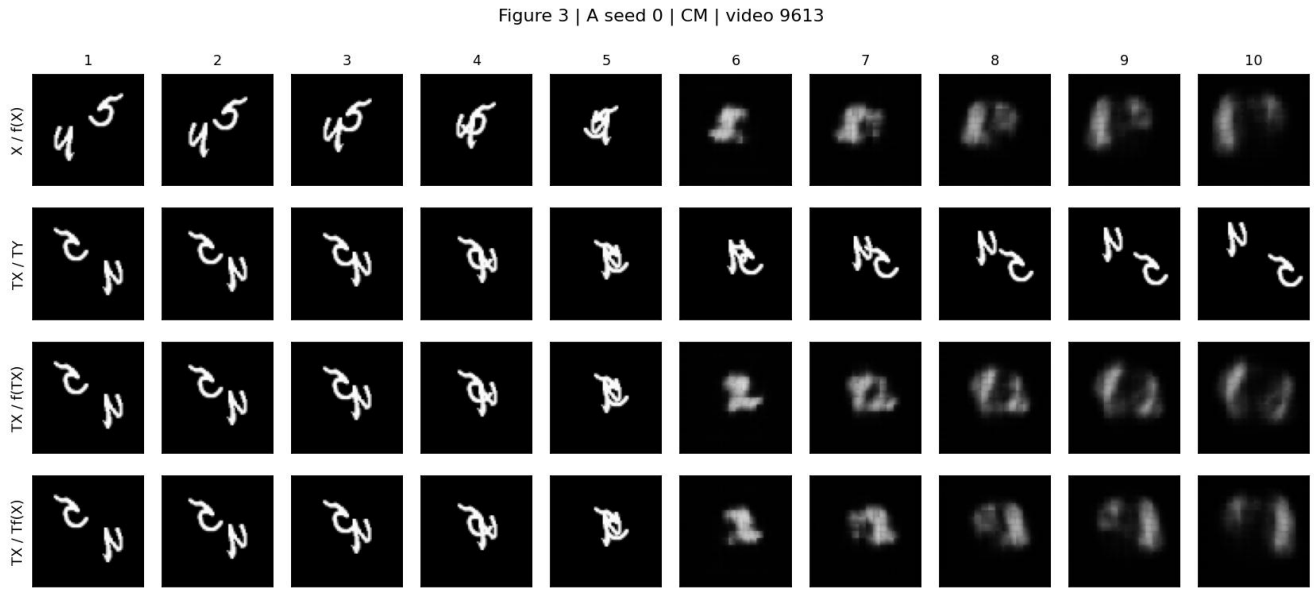


Figure 3, mirror case 1. A seed 0, test video 9613, 1×. Rows follow the four-route layout described above. GIF: outputs/figure3_CM_1_video9613.gif.

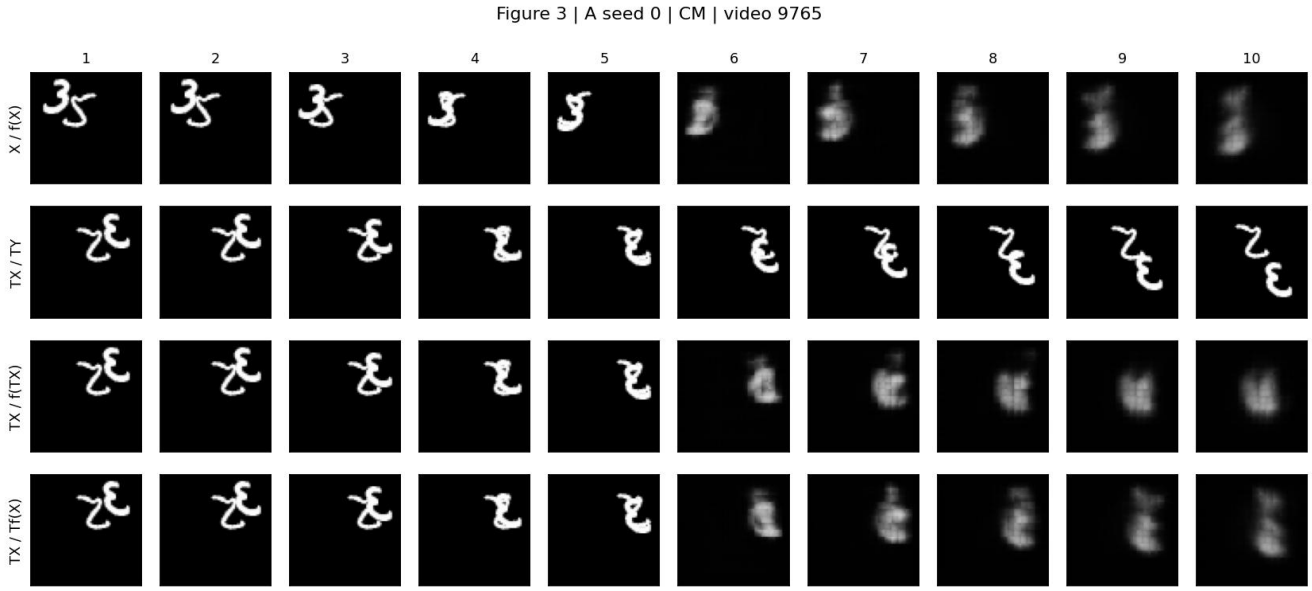


Figure 3, mirror case 2. A seed 0, test video 9765, 1×. Rows follow the four-route layout described above. GIF: outputs/figure3_CM_2_video9765.gif.

Both mirror examples contain interactions where the predicted digit shapes and their separation depend on whether mirroring occurs before or after forecasting (row 3 versus row 4). The true transformed futures in row 2 remain sharp; the model's outputs are blurred.

Figure 3. Largest rotation inconsistencies

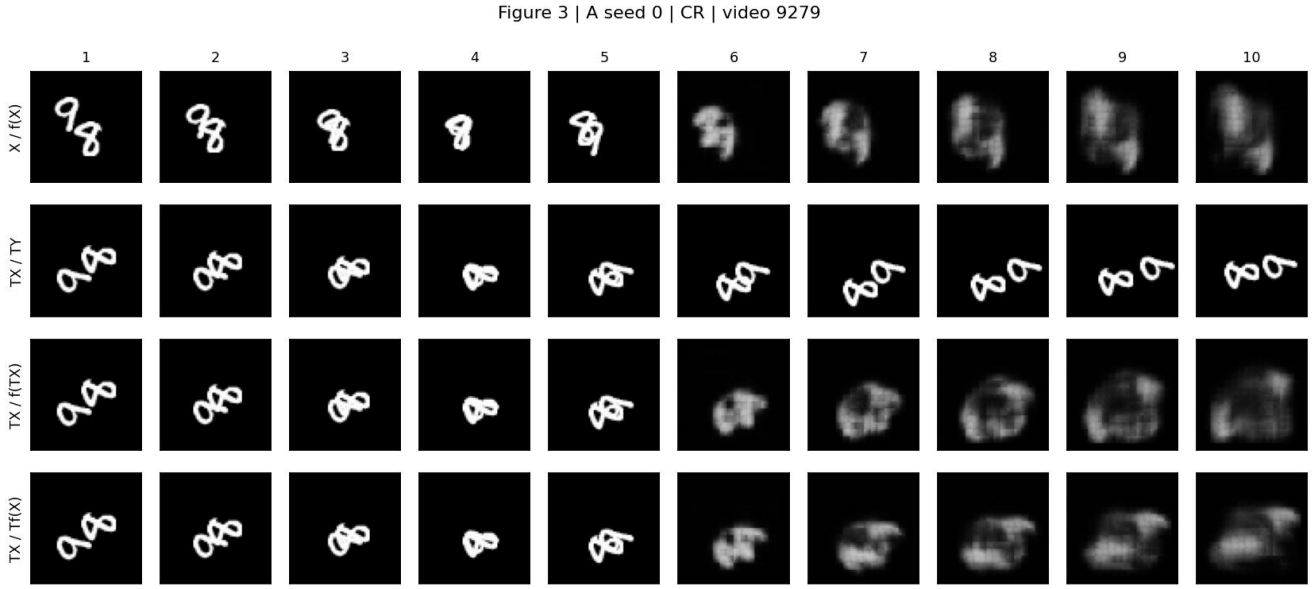


Figure 3, rotation case 1. A seed 0, test video 9279, 1×. GIF: outputs/figure3_CR_1_video9279.gif.

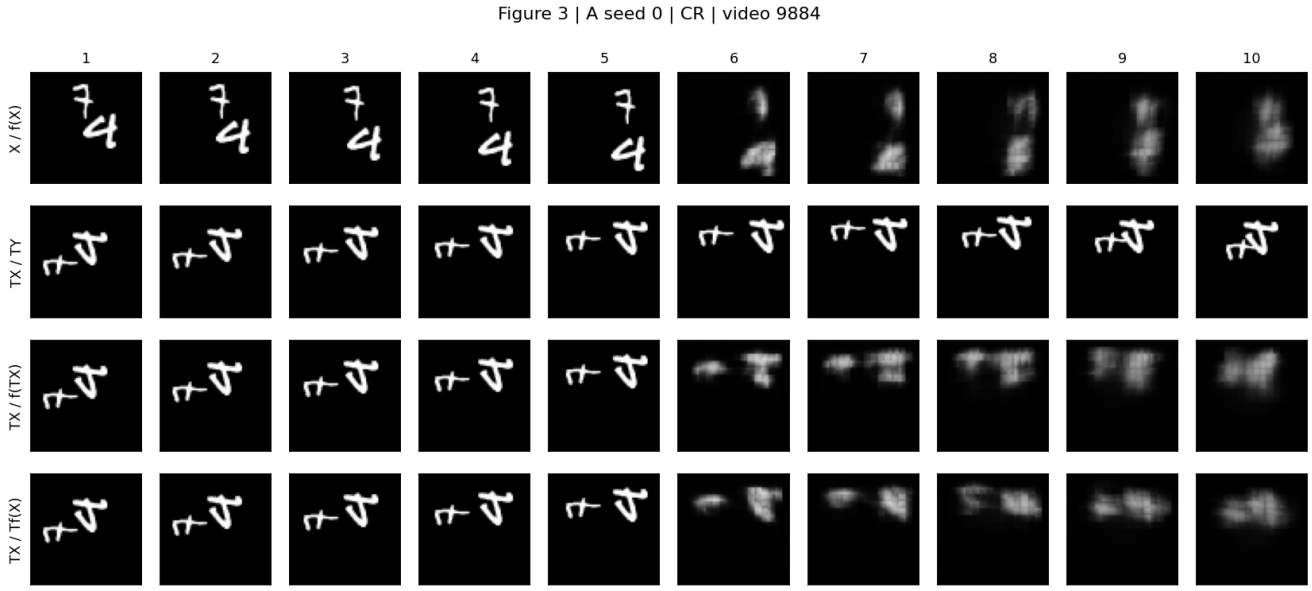


Figure 3, rotation case 2. A seed 0, test video 9884, 1×. GIF: outputs/figure3_CR_2_video9884.gif.

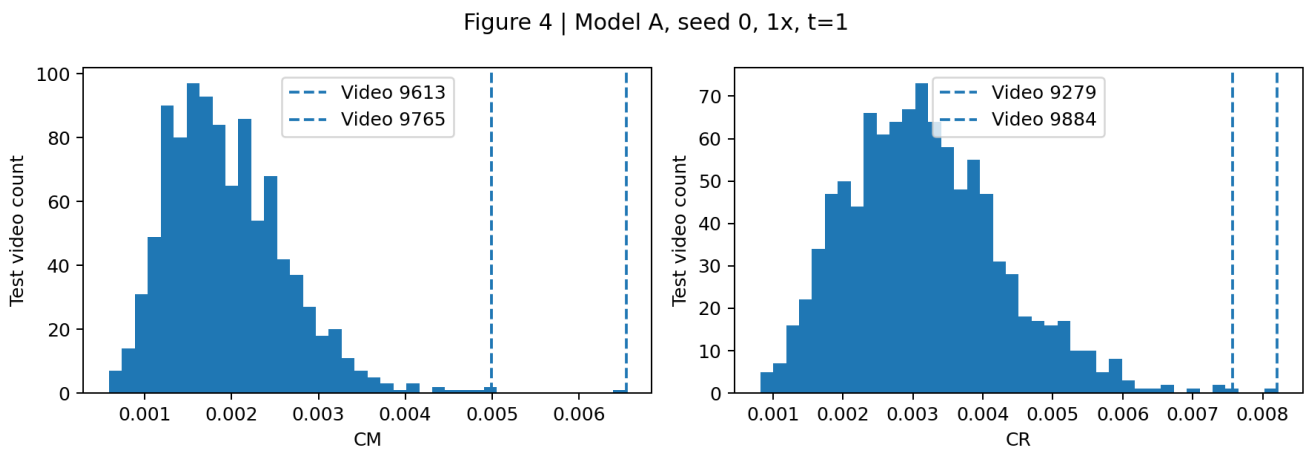


Figure 4. Per-video consistency gaps for A seed 0 over all 1,000 test videos. Dashed markers identify exactly the Figure 3 videos. These show the high-gap tail; overall correctness is reported alongside consistency in Table 3.

Q2.7

Video 9279, rotation, model A (seed 0). The true rotated future (row 2) shows an 8 and a 9 that overlap in frames 3–5 and then pull apart along a diagonal, the 8 towards the lower left and the 9 towards the upper right, both staying sharp. In row 3 (rotate, then forecast) the model does not separate them: it treats the overlapped pair as one object that stays where the overlap occurred and diffuses, by frame 10, into a broad, roughly round, textured blob with only a faint brighter patch at the upper right. In row 4 (forecast, then rotate) the blob is stretched along the horizontal axis, with a distinct brighter lobe at the upper right over a diffuse mass at the lower left, so that route begins to split the pair along roughly the correct axis. The model's account therefore changes from "the object stays and spreads" to "the object starts to split in the right direction". In neither row does a digit move the wrong way, vanish or bounce early; the failure is loss of shape and of separation, and it depends on orientation. Because the rows disagree at most one can be right, and by comparison with row 2 neither is.

Q2.8

Yes, rotation is harder for A, in both seeds. CR (0.00317, 0.00315) exceeds CM (0.00195, 0.00199), and ER (0.0241, 0.0236) exceeds EM (0.0230, 0.0225) in Table 3. By Q2.1 and Q2.2, both transformations remap the direction distribution and Figure 1(a) shows that every direction is already present in training, so neither presents motion the model has not seen. What differs most is appearance: rotation lays the digits on their side, mirroring only flips them. The extra difficulty therefore comes mainly from appearance, not from the motion. Supporting evidence: when symmetry augmentation teaches the appearances (model B, Table 5), the gap between the two consistency errors shrinks from 0.00119 (A: 0.00316 − 0.00197) to 0.00024 (B: 0.00145 − 0.00121). Two caveats: the training direction histogram is not exactly symmetric, so a small motion contribution cannot be excluded, and augmentation also rebalances directions, so the B comparison supports the conclusion without proving it.

4. Faster-motion stress tests

Table 4. Speed sensitivity of A and Copy-last

Metric	A	Copy-last
$E^{(2)}$, 1×	0.0187 (0.0190, 0.0184)	0.0548
$E^{(2)}$, 2×	0.0448 (0.0438, 0.0459)	0.0661
$E^{(2)}$, 3×	0.0499 (0.0496, 0.0502)	0.0703
$E^{(5)}$, 1×	0.0222 (0.0225, 0.0218)	0.0641
$E^{(5)}$, 2×	0.0493 (0.0486, 0.0500)	0.0719

Learned-model entries: mean (seed 0, seed 1). Copy-last is deterministic, so a single value is shown.

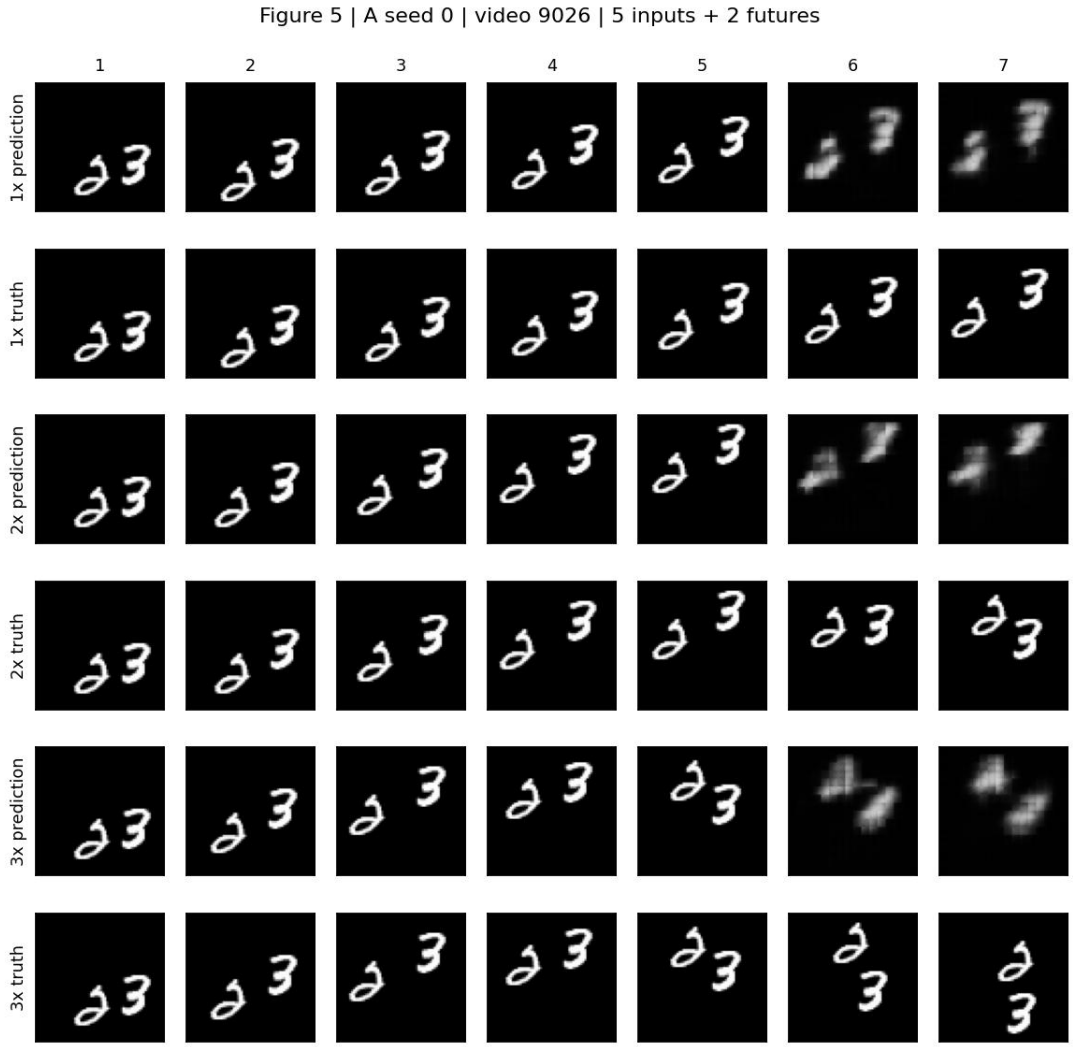


Figure 5. A seed 0 on test video 9026 at 1×, 2× and 3×. In each pair of rows, columns 1–5 show the inputs; columns 6–7 show the first two predicted or true future frames. Digits are separated in all displayed true frames.

Interpreting the speed failure

Q2.9

A degrades substantially already at 2×: its mean E(2) rises from 0.0187 at 1× to 0.0448 at 2× and 0.0499 at 3×. These are increases of approximately 2.40× and 2.67×. At 2× and 3×, A is worse than Blank on the same first-two-horizon metric (Blank: 0.0425 and 0.0426), although it remains better than Copy-last. Thus the failure is not confined to 3×.

Does this agree with the reach from Q2.3? Only partly. A reach of 8 px per axis predicts trouble once the step exceeds 8 px. By the median that happens at 3× (≈ 10 px) but not at 2× (≈ 6.6 px), so I expected a breakdown at 3× and a degraded but working model at 2×. The data show failure already at 2×. The estimate missed three things: (i) displacements are a distribution, not a median, and from Figure 1(b) on the order of a fifth of the pixels (the tail above about 4 px/frame) would exceed 8 px in a 2× clip; (ii) skipped frames can hide a wall bounce, a change A has never seen in a single step; and (iii) A was only ever trained on 1× steps, so reach is a necessary condition for tracking, not a guarantee. Which of these dominates cannot be read from A alone (Q2.10).

What the failure in Figure 5 looks like (video 9026, model A, seed 0, first two forecast frames). The failure is blur plus lag, not duplication or disappearance. At 1× both digits are predicted in about the right places, as smeared blobs. At 2× the true 3 reaches the top wall and turns back down while the true 2 moves up; the predicted 3 stays near the top and the predicted 2 hardly moves, so the model extrapolates straight lines and misses the bounce. At 3× the true digits end up stacked near the middle of the frame; the predicted lower blob is close, but the upper one stays to the left of the true 2, and both outlines have dissolved.

What each symptom suggests. Blur suggests the model hedges between possible positions under pixel MSE, spreading intensity instead of committing to one. A digit that stalls or lags suggests it is moved by something close to a typical 1× step rather than the larger step that was observed. These panels show no duplicated digit and no digit that vanishes, so those failure types are not claimed.

Q2.10

A alone cannot distinguish unfamiliar fast motion from insufficient architectural reach. Compare A with C while keeping the architecture fixed: adding speed augmentation tests whether exposure to 2×/3×windows helps. Then compare C with D under the same speed-augmented training protocol: a further gain at high speed supports a benefit from the architectural change. Use the same test videos and first two horizons, with both seeds. Any candidate architecture should be selected using validation data only.

These comparisons are not perfect causal isolation. C also uses the prescribed wider start-frame ranges and variable target lengths, while dilation changes spatial sampling as well as reach. Nevertheless, A→C separates the baseline architecture from the speed-augmented protocol, and C→D holds that training protocol fixed. The cleanest design is a 2×2 grid, reach (A's or D's) by training data (1× only or speed-augmented); its missing cell, D trained on 1× only, would show whether more reach alone helps without exposure to fast motion.

5. What can data fix?

Q2.11

The following is the prediction.

``` I expect B to reduce mirror and rotation errors by exposing it to transformed digit appearances, although augmentation cannot guarantee exact consistency. I expect C to improve speed generalization through examples of faster motion, while its one-step reach remains unchanged. I expect D to help where C remains limited by local communication, although its dilated sampling can miss fine details and none of these changes guarantees a gain.```

A: baseline, 1× only, no symmetry augmentation. **B:** baseline, 1× only, a uniform random element of the eight square symmetries for every window. **C:** baseline, uniform speed sampling from {1,2,3}, no symmetry augmentation. **D:** enlarged-reach architecture, the same speed augmentation as C, no symmetry augmentation.

Table 5. Controlled model comparisons

Metric	A	B	C	D
$E_{(5)}$, 1×	0.0222	0.0224	0.0271	0.0288
	(0.0225, 0.0218)	(0.0232, 0.0217)	(0.0277, 0.0266)	(0.0289, 0.0286)
C_M	0.00197	0.00121	8.43e-04	0.00104
	(0.00195, 0.00199)	(0.00102, 0.00139)	(8.10e-04, 8.76e-04)	(7.85e-04, 0.00129)
E_M	0.0228	0.0224	0.0273	0.0290
	(0.0230, 0.0225)	(0.0231, 0.0217)	(0.0278, 0.0269)	(0.0292, 0.0289)
C_R	0.00316	0.00145	0.00189	0.00180
	(0.00317, 0.00315)	(0.00131, 0.00159)	(0.00158, 0.00219)	(0.00151, 0.00210)
E_R	0.0239	0.0225	0.0282	0.0297
	(0.0241, 0.0236)	(0.0232, 0.0217)	(0.0286, 0.0278)	(0.0297, 0.0297)
$E_{(2)}$, 1×	0.0187	0.0189	0.0233	0.0264
	(0.0190, 0.0184)	(0.0197, 0.0181)	(0.0236, 0.0230)	(0.0265, 0.0263)
$E_{(2)}$, 2×	0.0448	0.0445	0.0291	0.0293
	(0.0438, 0.0459)	(0.0434, 0.0456)	(0.0293, 0.0288)	(0.0294, 0.0293)
$E_{(2)}$, 3×	0.0499	0.0515	0.0334	0.0315
	(0.0496, 0.0502)	(0.0500, 0.0530)	(0.0336, 0.0332)	(0.0313, 0.0316)
Parameters	480784	480784	480784	480784
Training s/epoch	14.0	14.7	13.8	20.5
	(14.0, 14.0)	(14.7, 14.7)	(13.8, 13.8)	(20.7, 20.2)
Inference s /	0.444	0.436	0.440	0.703
1,000 videos	(0.457, 0.432)	(0.448, 0.424)	(0.448, 0.432)	(0.720, 0.687)

Learned-model entries: mean (seed 0, seed 1). Parameter counts are exact and identical across seeds.

Training time is mean wall-clock seconds per epoch over all 30 epochs, excluding the one-time dataset download. Each seed runs on its own Tesla T4. Inference time is for all 1,000 original 1× test videos, batch size 32, under eval() and no_grad(), timed after each batch reaches the device. Times are measured values, not theoretical operation counts.
```

Q2.12

D uses the same two ConvLSTM layers, hidden width 64, patch folding factor 4, output head, initialisation and nine-step rollout as A. Only the gate convolutions change: both 3×3 convolutions use dilation 2 and zero padding 2 instead of dilation 1 and padding 1. No attention or additional layers are used.

Reach of D = 2 layers × 2 cells × 4 pixels = 16 pixels per axis, or $16\sqrt{2} \approx 22.6$ pixels diagonally. This strictly exceeds A's 8-pixel per-axis reach. Dilation changes spacing, not kernel parameter count.

Layer 1 has $4\times64\times(16+64)\times9+256 = 184{,}576$ parameters; layer 2 has $4\times64\times(64+64)\times9+256 = 295{,}168$; the output head has $16\times64+16 = 1{,}040$. Total: **480,784**, below the permitted 721,176.

Attribution and accuracy tradeoffs

Q2.13

The evidence supports the following qualified labels:

• **Appearance-related symmetry failure:** A→B reduces mean mirror and rotation consistency gaps by about 39% and 54%. Mean transformed errors also decrease, from 0.0228 to 0.0224 for mirror and 0.0239 to 0.0225 for rotation. B is less sensitive to orientation, although exact consistency is not achieved. The mirror-error gain is not uniform across seeds: seed 0 slightly worsens, while seed 1 improves.

• **Unfamiliar motion:** A→C reduces $E_{(2)}$ by about 35% at 2× and 33% at 3×. Both seeds improve. The same architecture can therefore handle substantially more of the fast-motion task once trained with the prescribed speed-augmented windows.

• **Architectural reach contributes at 3×:** C→D reduces mean 3× $E_{(2)}$ from 0.0334 to 0.0315, an improvement of about 6%, in the same direction for both seeds. But C already improves substantially and D is slightly worse at 2×. The actual outcome is a modest additional 3× benefit, not a clean "C fails, D succeeds"separation.

Training data can change weights, but it cannot add a missing edge to the within-step dependency graph.

If a prediction requires information outside that graph in one update, more examples cannot directly create the connection. Historical state can still encode motion cues, so this geometric argument alone does not prove the cause of every forecast error. D's sparse sampling and optimisation behaviour are also relevant.

Q2.14

There is a normal-speed tradeoff. Relative to A's $E_{(5)}$, B increases the mean error by about 1%, C by about 22%, and D by about 30%. B's cost is seed-dependent: seed 0 worsens and seed 1 slightly improves. C and D worsen original-speed accuracy in both seeds.

For a system evaluated mainly at 1×, these speed-robust models would be less attractive. For this assignment's robustness objective, C/D offer a meaningful reduction in faster-motion error. C is best at 2× and is faster than D; D is best at 3× before wrapping. Therefore larger reach is not uniformly preferable. With only two training seeds, these are descriptive comparisons, not claims of statistical significance.

6. Final model and exact symmetry

The final model is **symmetrised D**, as specified in the handout. Symmetrised B is included as the required comparison. Optional D+B was not trained. For any trained model f, define:

$$\mathbf{f_{sym}(X) = (1/8) \sum_{g\in G} g^{-1} f(gX).}$$

For a fixed $a\in G$, set $h=ga$. Right multiplication permutes G, and $g^{-1}=a\,h^{-1}$. Therefore $f_{sym}(aX)=(1/8)\sum_{h\in G} a\,h^{-1} f(hX)=a\,f_{sym}(X)$.

Pixel permutations are linear, so a passes through the average.

Taking a=M or R proves $C_M=C_R=0$ in exact arithmetic. Floating-point summation explains the tiny measured residuals in Table 6.

Table 6. Symmetrised models

Metric	symB	symD
$E_{(5)}$, 1×	0.0219	0.0287
	(0.0227, 0.0211)	(0.0289, 0.0284)
C_M	8.43e-17	5.31e-17
	(7.64e-17, 9.23e-17)	(4.97e-17, 5.64e-17)
E_M	0.0219	0.0287
	(0.0227, 0.0211)	(0.0289, 0.0284)
C_R	6.81e-17	4.12e-17
	(6.21e-17, 7.41e-17)	(3.83e-17, 4.41e-17)
E_R	0.0219	0.0287
	(0.0227, 0.0211)	(0.0289, 0.0284)
$E_{(2)}$, 1×	0.0184	0.0264
	(0.0192, 0.0177)	(0.0265, 0.0263)
$E_{(2)}$, 2×	0.0437	0.0292
	(0.0428, 0.0446)	(0.0293, 0.0292)
$E_{(2)}$, 3×	0.0504	0.0313
	(0.0492, 0.0515)	(0.0311, 0.0314)
Parameters	480784	480784
Training s/epoch	14.7	20.5
	(14.7, 14.7)	(20.7, 20.2)
Inference s /	3.17	5.21
1,000 videos	(3.20, 3.15)	(5.28, 5.14)

Learned-model entries: mean (seed 0, seed 1). Parameter counts are exact and identical across seeds.

Symmetrised D is the final model, so it is not duplicated as a third column. Training times and parameter counts belong to the underlying B/D models; wrapping adds no training or learned parameters. Inference includes all eight transformations, forward passes, inverse transformations and averaging.

Q2.15

B learns improved consistency from augmentation, but its gaps remain nonzero: C_M=0.00121 and C_R=0.00145. Symmetrised B and D have gaps of order 10^{-17}, matching the exact group-averaging argument up to floating-point precision. Their original, mirrored and rotated errors agree to the reported precision because the predictor is equivariant and the transformations preserve squared error.

Wrapping B reduces original $E_{(5)}$ by about 2% (from 0.0224 to 0.0219), and reduces both faster-speed errors. Symmetrised B has the lowest original-speed error among the evaluated models, but still performs poorly at 3× (0.0504). Wrapping D yields smaller mean improvements: under 1% in each case (about 0.3% in original E(5), 0.3% at 2× and 0.6% at 3×). These are small aggregate gains, not uniformly improved individual predictions.

The cost is substantial: B inference increases from 0.436 to 3.17 seconds, or 7.27×; D increases from 0.703 to 5.21 seconds, or 7.40×. These are measured ratios for all 1,000 videos. Eight forwards need not produce exactly eight times the measured wall time.

Averaging can blur sharp predictions when the inverse-transformed forecasts disagree in position or appearance. In that situation, an average may be worse than an accurate single orientation. Here wrapping does not hurt the reported mean original $E_{(5)}$ or cross-speed $E_{(2)}$ values for B/D, but Figure 6 visibly remains diffuse and patterned. Improved MSE does not establish sharper or physically correct forecasts.

Figure 6(a). Final model on mirror cases

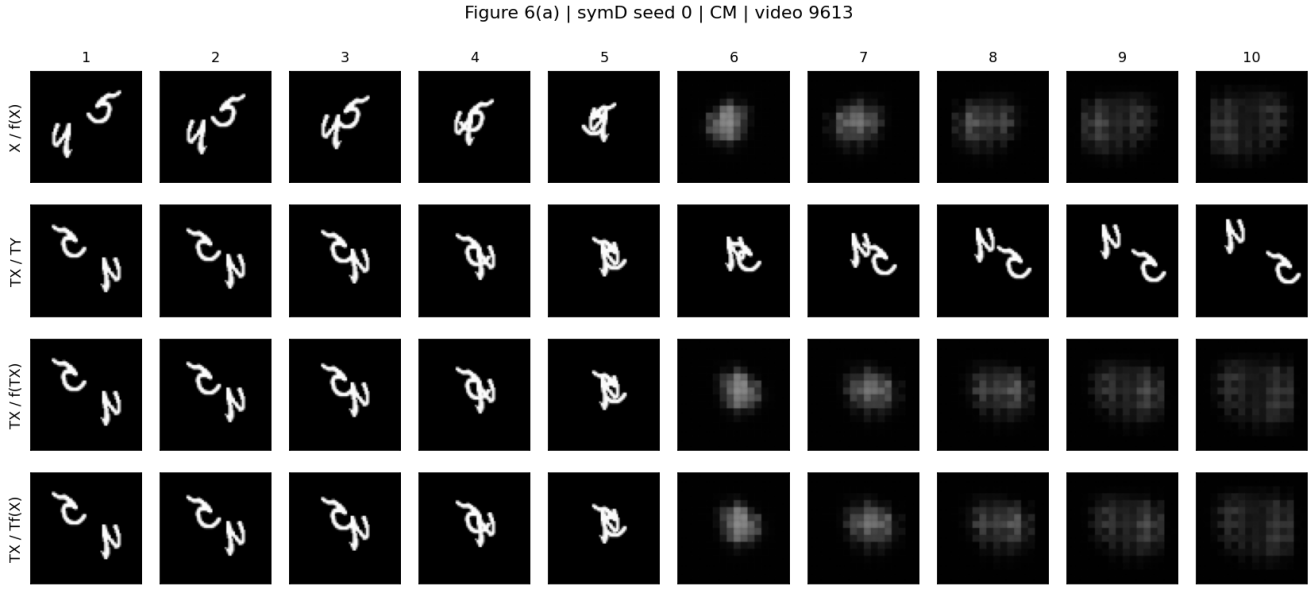


Figure 6(a), mirror case 1. Symmetrised D, seed 0, test video 9613, 1×. Same video and four-row layout as Figure 3. GIF: outputs/figure6a_CM_1_video9613.gif.

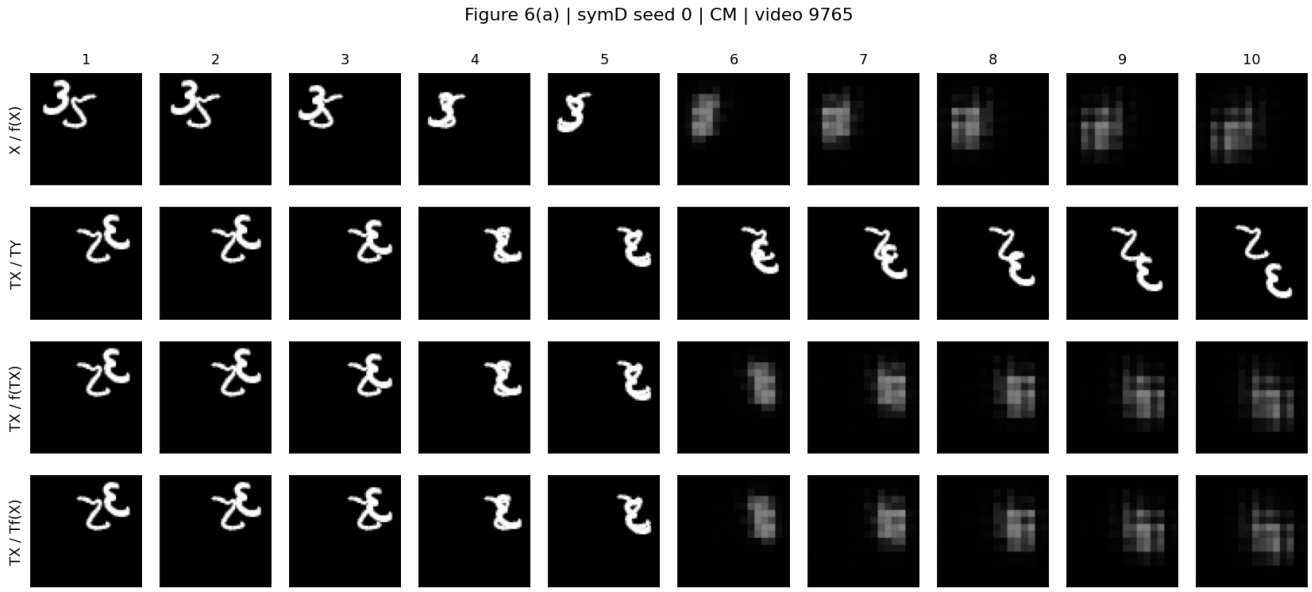


Figure 6(a), mirror case 2. Symmetrised D, seed 0, test video 9765, 1×. Same video and four-row layout as Figure 3. GIF: outputs/figure6a_CM_2_video9765.gif.

Rows 3 and 4 now agree, as expected from exact symmetrisation. Their agreement does not remove the blur or recover the sharply separated true digits in row 2.

Figure 6(a). Final model on rotation cases

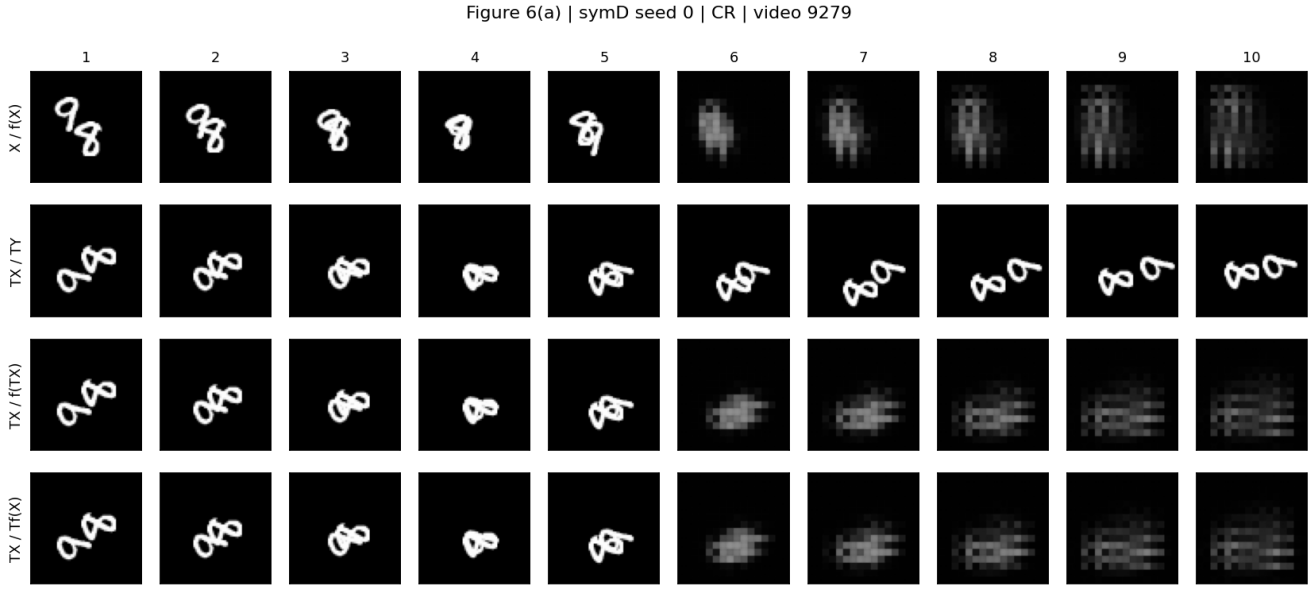


Figure 6(a), rotation case 1. Symmetrised D, seed 0, test video 9279, 1×. Same video and layout as Figure 3. GIF: outputs/figure6a_CR_1_video9279.gif.

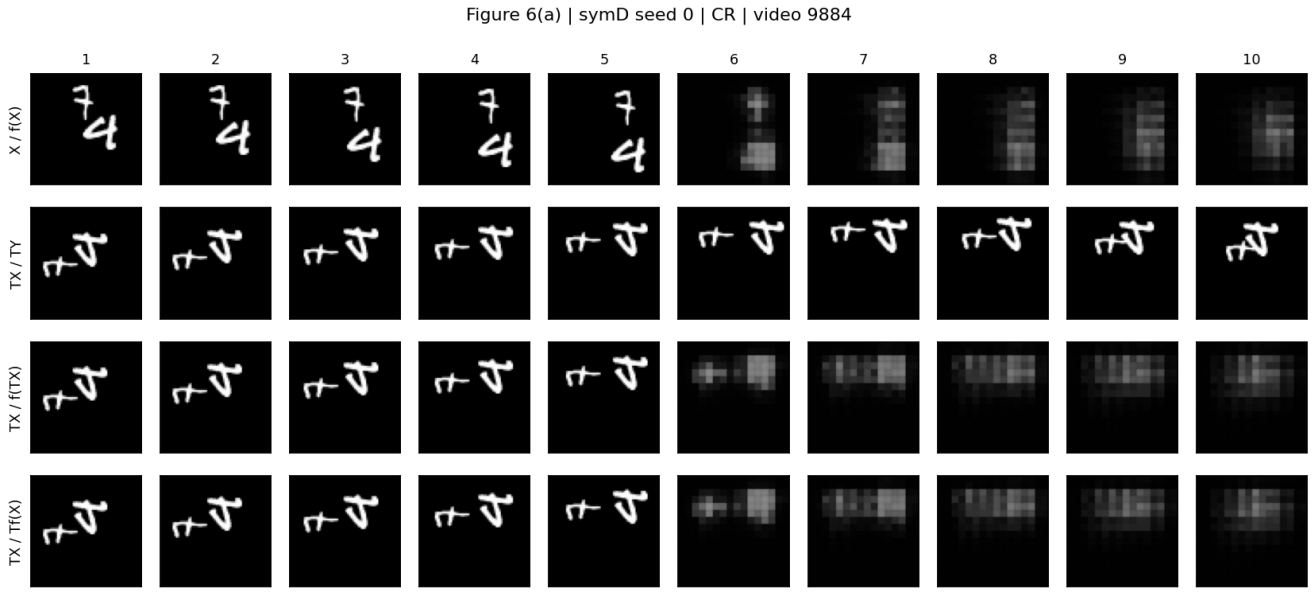


Figure 6(a), rotation case 2. Symmetrised D, seed 0, test video 9884, 1×. Same video and layout as Figure 3. GIF: outputs/figure6a_CR_2_video9884.gif.

Q2.16

Returning to video 9279 from Q2.7, the final model removes the disagreement between rows 3 and 4. It does not fully get the future right: both routes produce a broad, diffuse and patterned region, while row 2 contains two sharp digits separating horizontally. Thus the symmetry failure is fixed, but object identity and precise position remain unresolved.

The exact agreement is attributable to group averaging. Across the test set, A→C supports speed-exposure benefits and C→D supports an additional 3× architectural benefit. Those comparisons do not prove which component improves this particular 1× clip, and the final model is worse than A on mean original-speed error. Symmetry and accuracy must remain separate claims.

Figure 6(b). Final model at different speeds

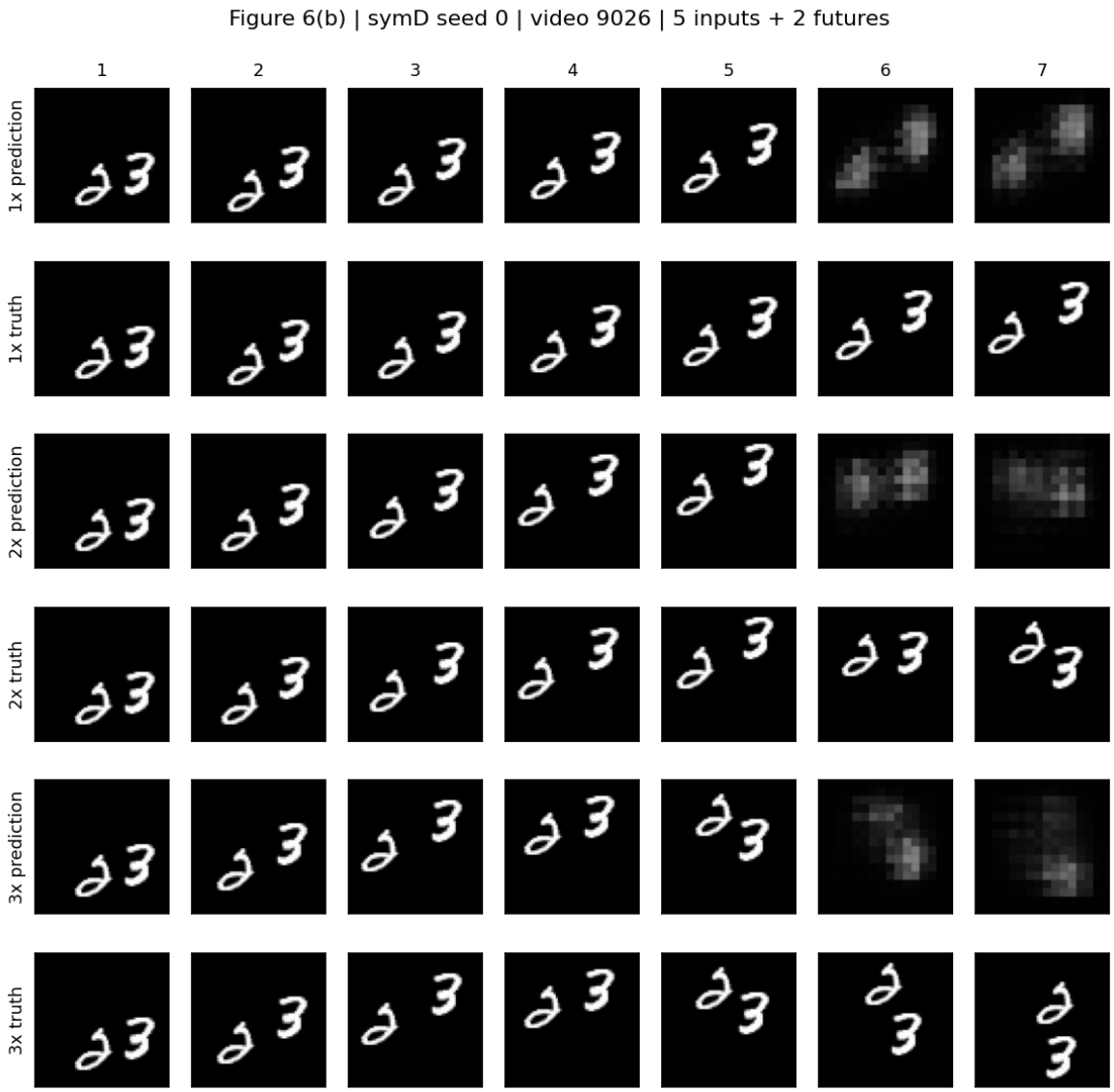


Figure 6(b). Symmetrised D, seed 0, on test video 9026, exactly the Figure 5 example. Five inputs and the first two future frames are shown at each speed. The true digits remain separated in the displayed frames. At faster speeds, the final forecasts remain diffuse; the aggregate speed gains do not imply sharp digit reconstruction.

Q2.17

The final model learned useful motion regularities, but not a complete motion rule. Against A, symmetrised D cuts first-two-horizon error by about 35% at 2× and 37% at 3×, and it is consistent under mirror and rotation to floating-point precision. But its original-speed E(5) is about 29% worse than A's, and its forecasts stay blurred and often fail to separate digits. Consistency is guaranteed by averaging, so it is no evidence about the base network. A skeptic should see both seeds, transformed errors next to the gaps, matched-video forecasts and the A/C/D comparisons. One test it has not passed: accelerating trajectories, where constant-velocity training may not suffice.